

Mini-Project (ML for Time Series) - MVA 2025/2026

Victorin Turnel victorin.turnel@ens-paris-saclay.fr
Adèle Dejoie adele.dejoie@gmail.com

July 8, 2026

1 Introduction & contribution

Anomaly detection plays a crucial role in monitoring critical systems, yet the persistent challenge of accessing labeled data necessitates models capable of identifying deviations without explicit ground truth. Addressing this, the [1] method utilizes a Transformer-based architecture with self-attention to capture temporal context, employing a self-supervised "data degradation scheme" that trains the model to detect synthetic outliers generated by perturbing healthy data. Since synthesizing effective anomalies requires a deep understanding of signal dynamics to create relevant perturbations, this report aims to apply this model to the PSM dataset and enhance the approach by integrating a more profound analysis of signal behavior while maintaining the self-supervised paradigm.

More specifically, this project presents four major contributions.

- First, we extend the original study by applying the AnomalyBERT method to the PSM dataset, which was not utilized by the authors.
- Second, to validate the relevance of deep learning and the learning of temporal context, we establish a benchmark comparing the model against detection methods covered in the course, namely statistical median, VAR, and Matrix Profile [2], accompanied by a specific analysis of evaluation metrics.
- Third, we perform an optimization of the degradation scheme on the new dataset, a process specifically focused on optimizing the injection probabilities of the synthetic noises used in the data degradation scheme.
- Finally, we introduce custom-designed synthetic noises tailored to the PSM dataset to further enhance detection performance.

Regarding the division of labor, the drafting of the report was shared equally between the members. Victorin Turnel was responsible for Task 2 (benchmark and implementation of course techniques) and Task 4 (improvements for the PSM dataset), while Adèle Dejoie managed Task 1 (identifying a relevant dataset suitable for the method) and Task 3 (optimization of the degradation scheme). In terms of code reuse, Task 1 utilized the base code without modification. Conversely, Task 2 required the re-implementation of the three selected course methods using Numpy and PyTorch, as well as a from-scratch re-implementation of the Recall, Precision, and F1-score metrics based on the original code logic. Task 3 did not require re-implementation as the optimization of the degradation scheme was empirical, whereas Task 4 involved the implementation of two novel data degradation techniques which were integrated into the `train.py` file.

2 Method

2.1 Mathematical Formalization and Pipeline

The proposed method operates on a multivariate time series window formally denoted as $X = x_{t_0:t_1} \in \mathbb{R}^{N \times D}$, where N is the window size and D represents the number of variables. The workflow follows a specific self-supervised pipeline designed to learn context without manual labeling.

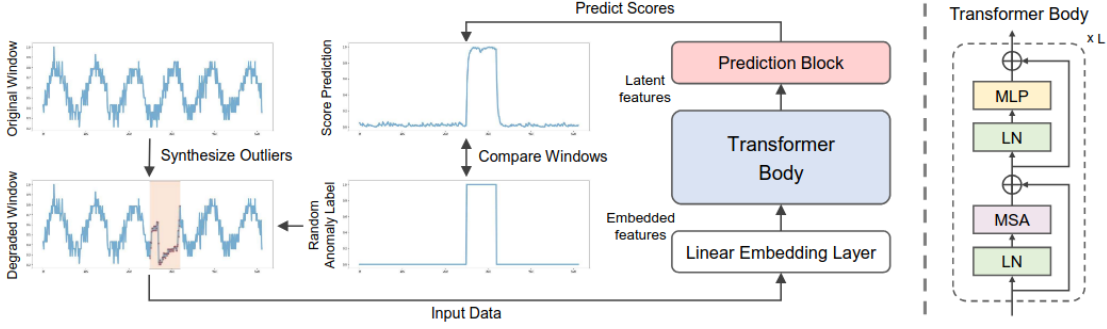


Figure 1: An overview of the AnomalyBERT framework

First, the original signal X undergoes a *data degradation scheme*. A specific interval $[t'_0, t'_1]$ within the window is selected and intentionally perturbed using synthetic noise (such as soft replacement or peak noise). This process creates a "degraded" input while simultaneously generating a precise ground truth mask indicating the location of the artificial anomaly. Second, this degraded signal is fed into a Transformer-based architecture. The model processes the data to understand the temporal context and predicts a sequence of anomaly scores. Finally, these predicted scores are compared against the generated ground truth mask to compute the error and optimize the model parameters.

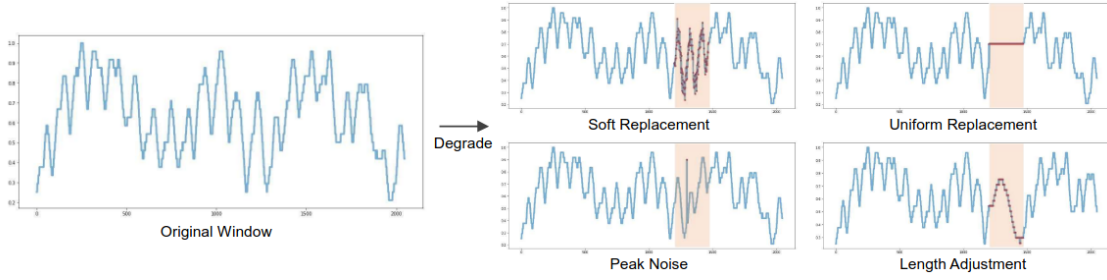


Figure 2: The four degradation types

To ensure the model learns diverse anomaly patterns, the degradation scheme employs four distinct types of synthetic outliers applied to the selected interval $[t'_0, t'_1]$. **Soft Replacement** substitutes the sequence with a weighted combination of the original data and an external segment to introduce subtle contextual anomalies. **Uniform Replacement** replaces the interval with a constant value, whereas **Peak Noise** injects a single extreme value to simulate sharp point outliers. Finally, **Length Adjustment** modifies the time-scale by stretching or compressing the segment, effectively mimicking frequency or speed deviations. Crucially, the occurrence probability of each perturbation type is configurable, allowing the training distribution to be tailored to specific dataset characteristics. Collectively, these perturbations allow the model to cover a broad taxonomy of anomalies, including global, contextual, and seasonal deviations.

Regarding the internal architecture, the signal is first processed by a linear embedding layer that projects local data patches $x_{t:t+p}$ into a latent space, producing a sequence of features f_i . These features are processed by the Transformer body. Unlike standard Transformers that utilize absolute positional encodings, AnomalyBERT integrates temporal information directly into the attention mechanism via an **ID relative position bias**.

Mathematically, for each attention head, the *Self-Attention* calculation is modified as follows:

$$Attention(Q, K, V) = \text{Softmax} \left(\frac{QK^T}{\sqrt{d}} + B \right) V \quad (1)$$

where Q , K , and V are the *Query*, *Key*, and *Value* matrices respectively, and d is the feature dimension. The term $B \in \mathbb{R}^{M \times M}$ represents the relative position bias matrix, where each element $b_{i,j}$ is a learnable parameter depending solely on the relative distance $j - i$ between two tokens. This allows the model to effectively capture local and global temporal dependencies without relying on fixed absolute positions.

The optimization process relies on minimizing the difference between the predicted anomaly scores and the artificial labels created during the degradation step. The objective is to predict an anomaly score $a_t \in [0, 1]$ for each time point t . The loss function to be minimized is the Binary Cross Entropy, defined by:

$$\mathcal{L} = -\frac{1}{N} \sum_{t=t_0}^{t_1} \left(\mathbb{1}_{[t'_0, t'_1]}(t) \cdot \log(a_t) + (1 - \mathbb{1}_{[t'_0, t'_1]}(t)) \cdot \log(1 - a_t) \right) \quad (2)$$

Here, the indicator function $\mathbb{1}_{[t'_0, t'_1]}(t)$ acts as the artificial label, taking the value 1 if time t falls within the degraded interval, and 0 otherwise. This formulation forces the network to precisely discriminate degraded segments from the original signal based on the global temporal context reconstructed by the Transformer.

3 Data

The **Pooled Server Metrics (PSM)** dataset (Abdulaal et al., 2021) [3] serves as a robust benchmark for multivariate anomaly detection. Collected from eBay’s production nodes, it aggregates 26 distinct dimensions—such as CPU utilization, memory usage, and network traffic—spanning approximately 21 weeks.

For this study, we utilized a pre-processed version of the dataset to ensure consistent data quality. The preprocessing pipeline involved two key steps:

- **Linear Interpolation:** Applied to impute missing values, ensuring temporal continuity without introducing artificial artifacts.
- **Normalization:** All dimensions were normalized to eliminate magnitude disparities between sensors, preventing features with large values from dominating the gradient descent.

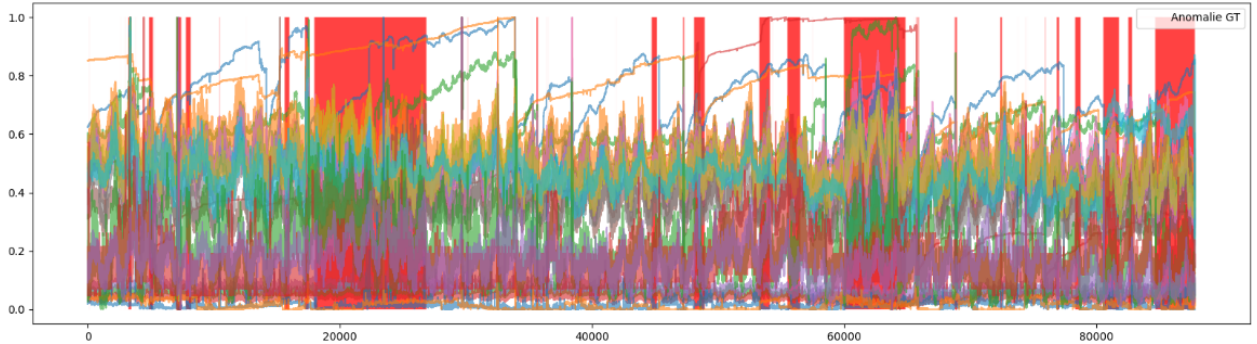


Figure 3: Visualization of the Ground Truth anomalies in the PSM test set.

The dataset is partitioned into a training set of 132,481 timestamps and a testing set of 87,841 timestamps. A defining characteristic of PSM is its exceptionally high anomaly rate of **27.76%** within the test set, which is significantly higher than other standard benchmarks like SMD (4.16%) or SWaT (12.1%). Crucially, the training set is guaranteed to be free of anomalies, fulfilling the requirement for self-supervised training on "normal" data.

Given that this study centers on the application of AnomalyBERT, detailed data diagnostics are presented in Appendix A rather than in the main report.

4 Experiments & results

4.1 Vanilla version on PSM Dataset

The initial phase of our experimental workflow involved training the model using the default hyperparameter configuration on the Pooled Server Metrics (PSM) dataset. The primary objective of this step was to gain practical familiarity with the training pipeline, specifically regarding the convergence behavior of the loss function and the computational resources required.

By deploying the "vanilla" version without modification, we aimed to establish a reliable baseline against which the impact of our subsequent optimization of the degradation scheme and architectural modifications could be measured. This preliminary run also allowed us to verify the reproducibility of the training loop and ensure the stability of the environment before introducing custom noise injection strategies or comparing the Deep Learning approach against statistical baselines. More details about training can be found in appendix B.

4.2 Comparison with classical methods for anomaly detection

In a second phase, we investigated the relevance of employing complex deep learning architectures, such as Transformers, to capture temporal context in anomaly detection tasks. To assess whether such computational complexity is justified, we implemented three statistical algorithms covered in the course using NumPy and PyTorch: the adaptive Median/Median Absolute Deviation (MAD), Vector Autoregression (VAR), and the Matrix Profile. We then conducted a comparative analysis of these classical approaches against the Anomaly Transformer on the test partition of the PSM dataset.

To ensure a rigorous comparison with the probabilistic outputs of the deep learning model, we adapted the classical algorithms to generate continuous anomaly scores rather than discrete binary classifications. These scores were normalized to the $[0, 1]$ range using a sigmoid function, aligning them with the Anomaly Transformer's output scale. Furthermore, following standard evaluation protocols for self-supervised anomaly detection, we adopted a dynamic thresholding strategy based on the known anomaly prior. Specifically, we determined the decision threshold via quantile analysis to force the model's detection rate to match the ground truth anomaly ratio of the PSM dataset (27.8%), thereby evaluating the theoretical maximum performance of each method.

Table 1: Performance comparison on PSM Dataset : **MAD** : the adaptive Median/Median Absolute Deviation, **VAR** : Vector Autoregression, **MP** : Matrix profil

Méthode	Precision	Recall	F1-score
AnomalyBERT	0.48372	0.50746	0.49513
MAD	0.3815	0.3848	0.3832
VAR	0.3557	0.3588	0.3572
MP	0.1936	0.1953	0.1945

As illustrated by the results, the AnomalyBERT model significantly outperforms the traditional approaches. The moderate performance of the Median Absolute Deviation (MAD) method can be attributed to its reliance on detecting abrupt deviations in statistical dispersion. In the context of the PSM dataset, high variability is often regular behavior (servers starting), whereas anomalies frequently manifest as periods of excessive regularity (e.g., signal stagnation due to CPU saturation), which MAD fails to capture. Regarding the Vector Autoregression (VAR) model, it also achieved moderate performance, while it successfully models linear inter-dependencies between sensors, it is limited by its inability to capture the complex non-linear temporal dynamics inherent in server telemetry. Finally, the Matrix Profile (MP) yielded the lowest scores. Fundamentally, MP identifies anomalies as "discords"—unique subsequences lacking near neighbors. However, in this dataset, anomalous patterns tend to recur multiple times. Consequently, these repeated faults are not recognized as singular events by the algorithm, leading to a high rate of false negatives.

4.3 Optimization of the degradation scheme on PSM Dataset

In this section, we propose a new experiment that consists in fine-tuning the frequency of occurrence for each perturbation during the training phase. Since we can control the probability of each noise type, it is important to select only those that are relevant to the real signal. Indeed, some synthetic perturbations may differ too much from reality and confuse the model. To evaluate the impact of the four available perturbations on the PSM dataset, we adopted the following protocol: for each experiment, we fixed the probability of the target perturbation to 0.5, while keeping the others at a baseline of 0.1. We repeated this process for all four types. We then compared the strategies based on their performance on the test set. These results will allow us to determine a better weighting for each perturbation in the final training configuration.

Table 2: Sensitivity analysis of noise injection strategies: Dominant ($p=0.5$) vs Baseline ($p=0.1$)

Perturbation	Precision	Recall	F1-score
Soft replacing	0.45738	0.49436	0.47515
Uniform replacing	0.41849	0.45232	0.43475
Peak noising	0.40972	0.44284	0.42564
Length adjusting	0.31088	0.33784	0.32574

As observed, the **Length Adjustment** perturbation leads to poor performance. This can be explained by the fact that server metrics typically follow a fixed sampling frequency. Therefore, time-stretching artifacts do not reflect realistic anomalies. **Uniform Replacement** yields good results because it mimics resource saturation, such as a CPU reaching a ceiling value with low variance. However, this noise remains too simple to fully capture complex system behaviors. Unsurprisingly, **Soft Replacing** proves to be the most effective perturbation. This is because it preserves the local statistical properties of the signal while breaking the temporal context, which closely resembles realistic contextual anomalies. Finally, **Peak Noise** provides reasonable performance since some faults manifest as spikes (sudden traffic spikes, CPU spikes). However, as demonstrated in the previous section with the MAD approach, the majority of real anomalies in this dataset do not rely on abrupt signal variations.

For the final experiment, we compared the performance of two models on the test set: one trained with the **default configuration** and another trained with the **optimized weighting** scheme derived from our previous analysis. The model with adjusted weights achieved superior performance with a f1-score of 0.50561 compared to 0.49513 with the default configuration.

4.4 Improvements

In this section, we propose to enhance the current method by introducing two new types of perturbation. First, it is essential to understand the nature of anomalies in the PSM dataset. In this context, anomalies typically correspond to specific incidents such as memory leaks, CPU saturation, or frozen processes. Therefore, the first perturbation, namely **the Uniform Shift**, adds a constant value to a segment to simulate a sudden state change (e.g., a service crashing or starting). The second, **Linear Drift**, adds a progressive ramp to simulate a gradual degradation, such as a memory leak. The final experiment consists of testing whether these domain-specific perturbations improve the model’s performance.

Thanks to these enhancements, we achieved an improvement in generalization performance on the test set. A comparison with the default configuration reveals a slight increase in the F1-score (between 0.51204 and 0.53987 on several runs). Visual inspection further confirms that the model is now capable of detecting much finer anomalies, particularly those based on complex inter-signal dependencies. A striking example is a broken correlation scenario where one metric (e.g., CPU usage) gradually drifts upward while a correlated metric (e.g., Request Throughput) remains stable.

References

- [1] Y. Jeong, E. Yang, J. H. Ryu, I. Park, and M. Kang, "Anomalybert: Self-supervised transformer for time series anomaly detection using data degradation scheme," 2023. [Online]. Available: <https://arxiv.org/abs/2305.04468>
- [2] C.-C. M. Yeh, Y. Zhu, L. Ulanova, N. Begum, Y. Ding, H. A. Dau, D. F. Silva, A. Mueen, and E. Keogh, "Matrix profile i: All pairs similarity joins for time series: A unifying view that includes motifs, discords and shapelets," pp. 1317–1322, 2016.
- [3] A. Abdulaal and T. Lancewicki, "Real-time synchronization in neural networks for multivariate time series anomaly detection," IEEE, 2021.

A Data diagnosis

Appendix A: Multivariate Data Diagnosis (PSM Dataset)
Figure A.1: Correlation Matrix (Inter-Sensor Dependencies)

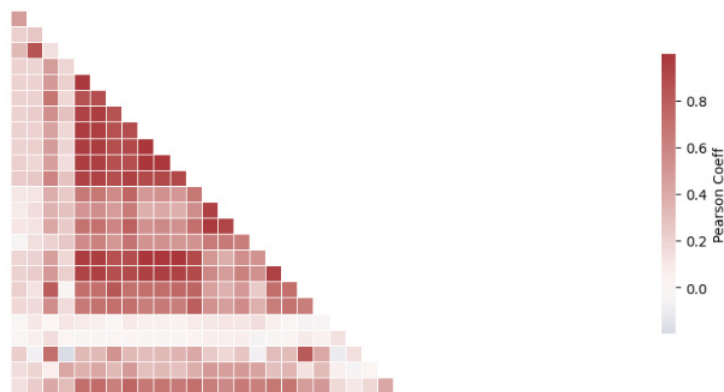


Figure A.2: Spectral Analysis (FFT) - Seasonality Check (Dim 13)

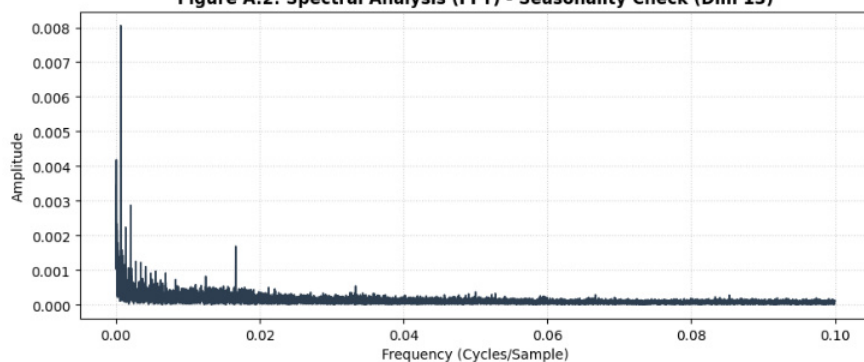


Figure A.3: Rolling Means of All 25 Signals (Trend & Non-Stationarity)

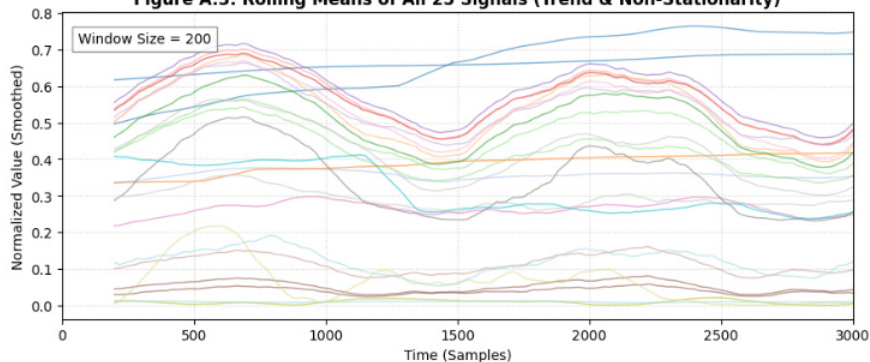


Figure 4: Data diagnosis

Figure A.1 reveals strong clusters of inter-variable dependencies. This high redundancy implies that anomalies may manifest not just as value spikes, but as broken correlations.

The FFT analysis on dimension 13 (Figure A.2) highlights a dominant low-frequency peak. Unlike white noise, which consists of all frequencies, here we observe dominant low frequencies. This confirms the strong seasonality of the server metrics (likely daily cycles).

Figure A.3 demonstrates distinct non-stationarity across all dimensions. The local mean of the signal isn't constant and evolves over time. This can be challenging for statistical methods like MAD that rely on constant statistical properties.

B Training on PSM Dataset details

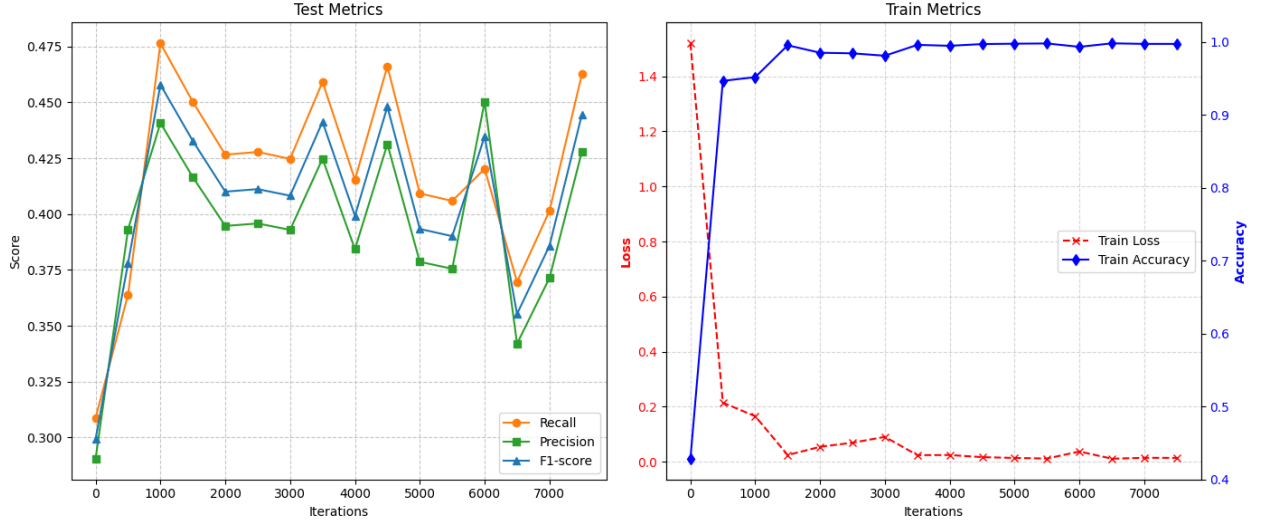


Figure 5: (Left) : Test metrics (Right) : Train metrics

The examination of the training and testing curves (Figure 5) highlights a significant gap between the model’s ability to learn the synthetic task and its performance on real anomalies.

We observe an extremely rapid convergence of the training metrics. The Training Accuracy reaches a saturation point of 100% (1.0), and the Loss drops to near-zero values within the first 1,000 iterations. This indicates that distinguishing synthetic perturbations from normal data is trivial for the model. It successfully separates the perturbed samples without difficulty. However, this "perfect" learning suggests that the model is likely memorizing the specific patterns of our synthetic artifacts (such as sharp peaks or flat segments) rather than learning the deeper, complex structure of the normal signal.

In contrast to the stability of the training phase, the metrics on the test set (F1-score, Precision, Recall) show high volatility. The curves oscillate significantly between evaluations. This instability reveals a lack of generalization: while the model is an expert at identifying our synthetic defects, it struggles to transfer this capability to the real-world anomalies of the PSM dataset. Furthermore, the peak performance is achieved very early (around 1,000 iterations), and prolonged training does not allow stable improvements. It confirms that the model saturates quickly.

There is a clear gap between the perfect training performance and the unstable testing results. This phenomenon is characteristic of overfitting to the proxy task. It suggests that our synthetic perturbations are likely too distinct compared to real anomalies, which are often more subtle. To improve results, it would be necessary to increase the difficulty of the training task, for example, by generating less obvious perturbations that force the model to learn more robust features.

C Code contribution details

For this project, we primarily utilized the code from the original repository: <https://github.com/Jhryu30/AnomalyBERT>. Given the complexity of the codebase, we highlight the key files and our specific contributions below.

Specifically, the following elements were modified or added:

- **Modification of train.py:** We added two new types of noise (lines 298–323).

- **Addition of `baseline.ipynb`:** This notebook contains our implementations of the methods studied in class, along with the evaluation metrics. It was developed entirely from scratch.
- **Addition of `my_test.ipynb`:** We created this notebook to test and evaluate the predictions from our trained models.